

Scuderia

May 15, 2026

Noțiuni semantice

1. Operator $\lceil n \rceil$ reprezintă aproximarea prin adaos la unități a numărului n :

Exemple: $\lceil 2.1 \rceil = 3$, $\lceil 2 \rceil = 2$

2. a este șirul cu n elemente din enunț, indexat de la 0.
3. Prin noțiunea de **consecutiv circular** în cadrul unui șir v de lungime l , indexat de la 0, înțelegem că succesorul elementului de pe poziția $l - 1$ este elementul de pe poziția 0. Astfel, orice indice i va fi tratat prin prisma valorii $i \% l$.

Exemplu: Pentru un șir s de lungime $k = 7$, elementele consecutive circular începând de la poziția 6 pentru o fereastră de lungime $p = 3$ sunt $s[6]$, $s[(6 + 1) \% 7]$ și $s[(6 + 2) \% 7]$, adică $s[6]$, $s[0]$ și $s[1]$.

Observații esențiale

1. Faptul că un indice j aparține mulțimii dacă și numai dacă indicele $j + k$ aparține aceleiași mulțimi, **ori de câte ori ambii indici sunt cuprinși între 0 și $n - 1$** , este echivalent cu: $j - k$ aparține mulțimii dacă și numai dacă indicele j aparține aceleiași mulțimi, **ori de câte ori ambii indici sunt cuprinși între 0 și $n - 1$** .

2. $j \% k = (j + k) \% k$

Subtask 1: $k = 1$ (10 puncte)

Singura valoare validă a lui r este 0. Dacă $p > 0$, Luis va selecta toate elementele din șir, și răspunsul va fi suma elementelor din șir. Dacă $p = 0, q > 0$, atunci răspunsul va fi valoarea maximă a sumei a q numere consecutive. Dacă $p = 0, q = 0$, atunci răspunsul va fi 0.

Subtask 2: $p = 0, n \leq 1\,000$ (8 puncte)

Dacă $p = 0$, doar Carol va selecta numere. Problema se transformă în a determina subsecvența maximă a $q \cdot k$ numere, iar indicele de început al subsecvenței să aibă restul împărțirii la $k = r$. Pentru fiecare indice valid, calculăm subsecvența într-un mod iterativ. Complexitatea finală e $O(q \cdot k \cdot \lceil \frac{n}{k} \rceil)$.

Subtask 3: $p = 0$ (15 puncte)

Soluția este extrem de similară cu cea a subtaskului 2. Însă, în loc să calculăm iterativ pentru fiecare valoare, vor reține un șir s de sume parțiale.

$s[i]$ -suma elementelor aflate între pozițiile 0 și i

Definim relația de recurență pentru șirul sumelor parțiale s astfel:

$$s[i] = \begin{cases} a[0] & \text{dacă } i = 0 \\ s[i-1] + a[i] & \text{dacă } i > 0 \end{cases}$$

Această construcție ne permite să calculăm suma elementelor din intervalul $[i, j]$ în timp constant, folosind formula:

$$\text{Suma}(i, j) = s[j] - s[i-1]$$

Pentru fiecare indice $i, i \% k = r$, indicele de final al subsecvenței va fi $i + q \cdot k - 1$. Dacă $i + q \cdot k - 1 < n$, atunci avem o subsecvență validă, a cărei sumă o vom calcula folosind metoda mai sus ($s[i + q \cdot k - 1] - s[i - 1]$). Complexitatea finală e $O(n)$, putând fi îmbunătățită la $O(\lceil \frac{n}{k} \rceil)$, dacă iterația pentru verificare poziției începe din start de la r și are pasul k , folosind, spre exemplu: `for (int i = r; i + q * k - 1 < n; i += k).`

Subtask 4: $q = 0, p = 1$ (13 puncte)

Pentru acest subtask, și pe baza observației de mai sus, trebuie să calculăm suma maximă a numerelor care se află la distanță k unul de celălalt. Pentru fiecare din primele k numere, mai facem o iterație având pasul k , și calculăm suma acestor numere. Dintre toate aceste numere, selectăm suma maximă. Complexitatea finală e $O(n)$.

Subtask 5: $q = 0, k \leq 1\,000$ (7 puncte)

Soluția extinde logica subtaskului precedent prin gruparea elementelor în funcție de restul indicilor la împărțirea cu k . Definim șirul sr de dimensiune k , unde fiecare element $sr[r]$, $0 \leq r < k$ reprezintă suma tuturor valorilor din șirul original care au indicii i , iar $i \% k = r$.

$$sr[r] = \sum_{i \% k = r} a[i], \quad \forall r \in \{0, 1, \dots, k-1\}$$

Conform observației făcute la început, problema se reduce la identificarea unei subsecvențe circulare de lungime p în șirul s care să maximizeze suma totală.

Foarte important: Datorită modului în care Luis face selecția, elementele alese pot proveni din resturi consecutive care „trec” de capătul șirului s . Spre exemplu, pentru $n = 10, k = 4$ și $p = 2$, putem selecta inițial valorile de pe pozițiile 3 și 4, corespunzătoare resturilor 3 și 0, iar la final, mulțimea de indici rezultați să fie $\{0, 3, 4, 7, 8\}$.

Așadar, trebuie de asemenea efectuată o verificare a consecutivității circulare a sumei pe șirul sr . Matematic, suma pentru o subsecvență de lungime p care începe la indexul i se definește prin relația de recurență circulară:

$$\text{Suma}(i, p) = \sum_{j=0}^{p-1} sr[(i + j) \bmod k]$$

Problema se reduce astfel la determinarea maximului acestei sume pentru $i \in \{0, 1, \dots, k-1\}$, procedeu similar cu cel descris la subtaskul 2, dar adaptat pentru structura circulară a resturilor. Complexitatea finală e $O(k \cdot p)$.

Subtask 6: $q = 0$ (12 puncte)

Repetăm procedeul de la subtaskul 5, însă reținem treptat suma a p valori consecutive din șirul de sume parțiale. După, iterând cu un indice i de la p până la $k + p - 1$ inclusiv, adăugăm elementul aflat la $sr[i\%k]$ și scădem elementul aflat la poziția $sr[(i - p)\%k]$. Dintre toate aceste valori, o reținem pe cea cu valoarea maximă. Complexitatea finală e $O(k + p)$.

Subtask 7: $q = 1, p = 1$ (7 puncte)

Fiind ambele valori 1, cu siguranță va exista **un singur element** care se află atât în mulțimea lui Carol, cât și în mulțimea lui Luis. Vom combina ideile de la subtaskurile anterioare.

Începem prin a calcula sumele parțiale pentru pozițiile corespunzătoare resturilor, asemenea subtaskului 5. După aceea, similar cu subtaskul 3, vom parcurge toate subsecvențele din k în k cu un indice i , începând de la r până la $n - k - 1$. Vom calcula suma pe subsecvența $i, i + 1, \dots, i + k - 1$, iar, pentru fiecare poziție j , $i \leq j \leq i + k$, vom adăuga $sr[j\%k]$, și vom scădea elementul aflat în ambele mulțimi, adică $a[j]$.

La final, reținem valoarea maximă care se poate obține astfel. Complexitatea finală e $O(n)$.

Ultimele subtaskuri

Pentru ultimele subtaskuri, trebuie să schimbăm perspectiva asupra șirului. Modul de construcție al celor două mulțimi ne sugerează să privim șirul ca fiind o matrice în care sunt în total n elemente.

Matricea are k coloane și $\lceil \frac{n}{k} \rceil$ linii. Chiar dacă $n \% k \neq 0$, putem adăuga pe ultimul rând valori de 0, doar trebuie să avem grijă când selectăm mulțimea lui Carol (vezi 2).

De asemenea, în cazul în care $r \neq 0$, putem să adăugăm $k - r$ elemente cu valoare 0 la începutul șirului. Astfel, în șirul virtual construit, condiția de selecție a lui Carol se transformă:

$$j \% k = r \implies (j + k - r) \% k = (r + k - r) \% k \iff (j + k - r) \% k = k \% k = 0$$

Condiția devine astfel echivalentă cazului în care $r = 0$. Mulțimea lui Luis nu este afectată de aceste artificii, suma ei fiind exact aceeași (0 nu adaugă nimic în plus sumei finale). Echivalent, suma lui Carol devine suma a q linii consecutive din matrice, iar suma lui Luis devine suma a p coloane consecutive circular din matrice. Fiecare indice i din șirul inițial corespunde perechii de indici $\{\frac{i}{k}, i \% k\}$ în matrice.

Pentru primul caz, reprezentat în **Figura 1**, dacă adunăm suma elementelor din dreptunghiul roșu cu suma elementelor aflate în dreptunghiul galben, vom avea suma elementelor aflate în dreptunghiul portocaliu de două ori. Pentru a afla răspunsul final, vom scădea suma elementelor aflate în dreptunghiul portocaliu o dată, având astfel suma elementelor aflate în reuniunea finală a celor doi piloți.

	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	8	9	10	11	12	13	14	15
2	16	17	18	19	20	21	22	23
3	24	25	26	27	28	29	30	31
4	32	33	34	35	36	37	38	39
5	40	41	42	43	44	45	46	47

Carol: Liniile 2, 3

Luis: Coloanele 4, 5, 6

Figure 1: Carol alege liniile consecutive $\{2, 3\}$, colorate în roșu. Luis alege coloanele consecutive $\{4, 5, 6\}$, colorate în galben. Zona portocalie reprezintă intersecția celor două selecții. Toate pătratele colorate vor face parte din selecția finală.

Pentru al doilea caz, reprezentat în **Figura 2**, o modalitate de calcul e să calculăm suma tuturor elementelor, din care să scădem suma celor două dreptunghiuri albe. Astfel, obținem suma tuturor elementelor aflate în reuniune.

	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	8	9	10	11	12	13	14	15
2	16	17	18	19	20	21	22	23
3	24	25	26	27	28	29	30	31
4	32	33	34	35	36	37	38	39
5	40	41	42	43	44	45	46	47
6	48	49	50	51				

Carol: Liniile 4, 5 **Luis: Coloanele 6, 7, 0**

Figure 2: Carol alege liniile consecutive $\{4, 5\}$, colorate în roșu. Luis alege coloanele consecutive **circular** $\{6, 7, 0\}$, colorate în galben. Zona portocalie reprezintă intersecția celor două selecții. Toate pătratele colorate vor face parte din selecția finală. **Linia 6 nu poate fi selectată de către Carol**, ea fiind incompletă

Diferența dintre subtaskuri constă în eficiența calculării sumelor din dreptunghiuri. Pentru restricțiile în care $n \leq 100$, putem adopta o abordare directă: pentru fiecare configurație validă, iterăm prin toate elementele dreptunghiului delimitat de colțul stânga-sus (i_1, j_1) și cel dreapta-jos (i_2, j_2) .

Fiecărei perechi de indici (i, j) din matrice îi corespunde elementul $a[i \cdot k + j]$ din șirul original. Astfel, suma elementelor din dreptunghi se calculează în timp liniar față de aria acestuia, rezultând o complexitate finală de $O(n^2)$.

Pentru subtaskurile rămase, unde restricțiile devin mai stricte, abordarea iterativă nu mai este eficientă. Optimizarea constă în utilizarea **sumelor parțiale pe matrice**, tehnică ce ne permite să calculăm suma oricărui dreptunghi în timp constant, $O(1)$, după o preprocesare de $O(n)$.

Deoarece dimensiunile matricii depind de valorile citite în timpul rulării, nu putem declara o matrice statică cu dimensiuni fixe. În schimb, vom simula structura bidimensională folosind un vector liniarizat de dimensiune n .

Astfel, orice element aflat la coordonatele (i, j) în matricea conceptuală va fi accesat în șirul nostru prin formula de indexare $i \cdot k + j$. Această metodă ne permite să aplicăm algoritmul de sume parțiale descris anterior fără a consuma memorie suplimentară inutilă și fără a depinde de dimensiuni predefinite la compilare. Pentru ușurință în citire, ne vom referi la acest vector ca

fiind o matrice în rândurile următoare. Prin $a(i, j)$ ne referim la $a[i \cdot k + j]$.

Definim matricea sumelor parțiale S , unde $S[i][j]$ reprezintă suma tuturor elementelor din dreptunghiul cu colțul stânga-sus la $(0, 0)$ și colțul dreapta-jos la (i, j) :

$$S[i][j] = \sum_{r=0}^i \sum_{c=0}^j a(r, c)$$

Relația de recurență pentru construirea eficientă a acestei matrici, este:

$$S[i][j] = a(i, j) + S[i-1][j] + S[i][j-1] - S[i-1][j-1]$$

cu cazurile de bază tratate corespunzător pentru indicii egali cu 0. Odată construită această matrice, suma elementelor din orice dreptunghi delimitat de (i_1, j_1) și (i_2, j_2) se obține instantaneu prin formula:

$$\text{Sumă}((i_1, j_1), (i_2, j_2)) = S[i_2][j_2] - S[i_1-1][j_2] - S[i_2][j_1-1] + S[i_1-1][j_1-1]$$

Astfel, complexitatea finală a algoritmului devine $O(n)$. Algoritmul final integrează toate sub-taskurile, restricțiile neinfluențând modul de calcul al matricii și al sumelor din dreptunghiuri.

Observații extra soluție: Există metode de a alocă matricea în timpul rulării, însă acestea nu vor obține 100 de puncte, depășind limita de memorie impusă problemei. Acest lucru se poate face fie utilizând librăria `vector`, fie utilizând operatorul `new`.